

Container Orchestration with Kubernetes using Minikube

 Author

Oluwaseun Osunsola

[LinkedIn](#)

Project Overview

This project demonstrates the setup and deployment of a local Kubernetes environment using **Minikube** and **Docker** on Ubuntu (VMware). It covers:

- Environment preparation
 - Docker installation and validation
 - Minikube cluster setup
 - Kubernetes CLI (kubectl) usage
 - Application deployment and exposure
 - Scaling and self-healing demonstration
-

Project Objectives

By the end of this project:

- Understand Kubernetes architecture and components
 - Deploy a local Kubernetes cluster using Minikube
 - Build and run containerized applications
 - Perform scaling and observe self-healing
-

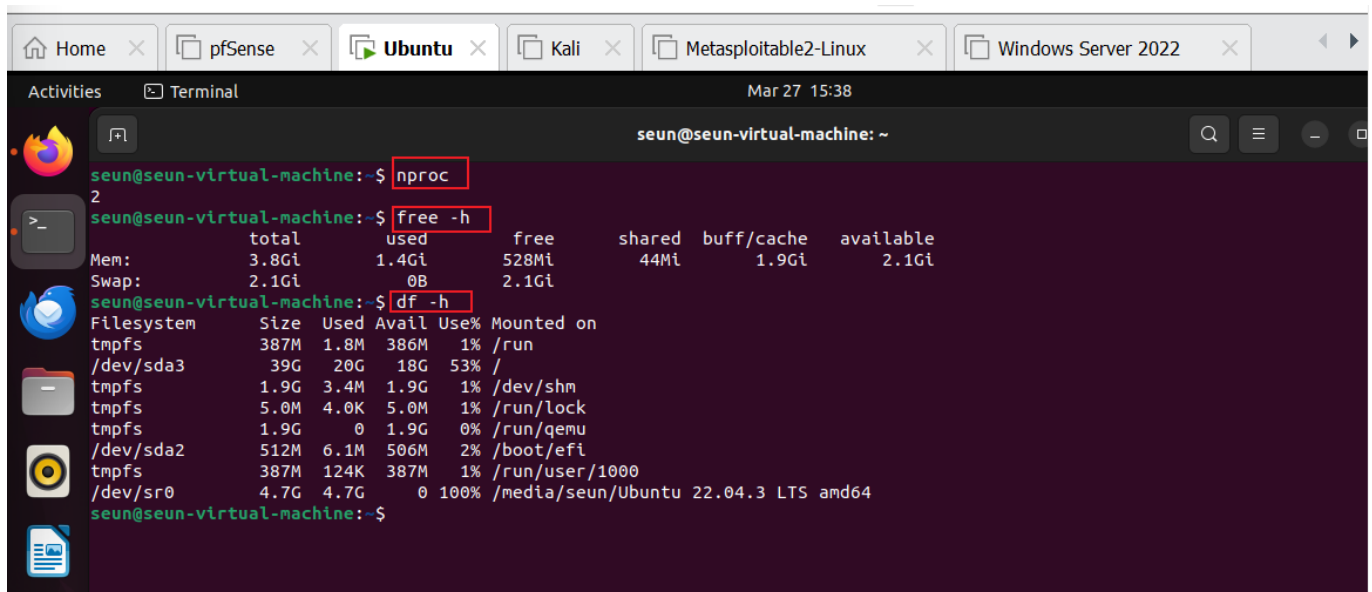
System Requirements

- CPU: ≥ 2 cores
 - RAM: ≥ 2 GB
 - Disk: ≥ 20 GB
-

PHASE 1: System Verification

Check available system resources:

```
nproc  
free -h  
df -h
```



The screenshot shows a terminal window titled 'seun@seun-virtual-machine: ~' with a dark purple background. The terminal displays the output of three commands: `nproc`, `free -h`, and `df -h`. The `nproc` command returns the value '2'. The `free -h` command shows memory usage statistics. The `df -h` command shows disk space usage for various filesystems.

```
seun@seun-virtual-machine:~$ nproc  
2  
seun@seun-virtual-machine:~$ free -h  
              total        used        free      shared  buff/cache   available  
Mem:           3.8Gi       1.4Gi       528Mi       44Mi        1.9Gi        2.1Gi  
Swap:          2.1Gi          0B        2.1Gi  
seun@seun-virtual-machine:~$ df -h  
Filesystem      Size  Used Avail Use% Mounted on  
tmpfs           387M  1.8M  386M   1% /run  
/dev/sda3       39G   20G   18G  53% /  
tmpfs           1.9G  3.4M   1.9G   1% /dev/shm  
tmpfs           5.0M  4.0K   5.0M   1% /run/lock  
tmpfs           1.9G   0     1.9G   0% /run/qemu  
/dev/sda2       512M  6.1M  506M   2% /boot/efi  
tmpfs           387M  124K  387M   1% /run/user/1000  
/dev/sr0        4.7G  4.7G   0 100% /media/seun/Ubuntu 22.04.3 LTS amd64  
seun@seun-virtual-machine:~$
```

◆ PHASE 2: Install Docker

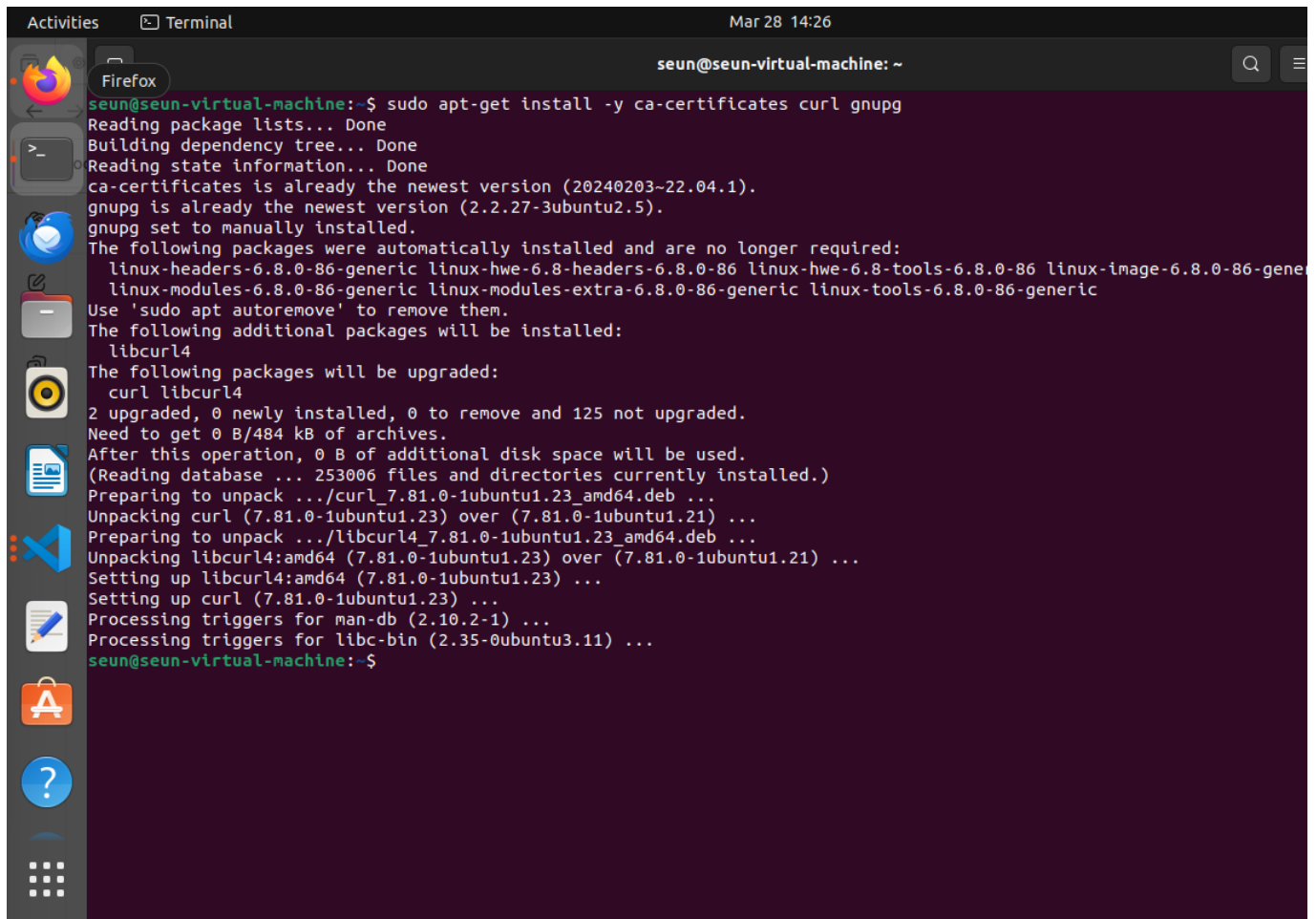
Step 1: Update package index

```
sudo apt-get update
```

```
seun@seun-virtual-machine:~$ sudo apt-get update  
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease  
Hit:2 http://ci.archive.ubuntu.com/ubuntu jammy InRelease  
Hit:3 https://packages.microsoft.com/repos/code stable InRelease  
Hit:4 http://ci.archive.ubuntu.com/ubuntu jammy-updates InRelease  
Hit:5 http://ci.archive.ubuntu.com/ubuntu jammy-backports InRelease  
Hit:6 https://download.docker.com/linux/ubuntu jammy InRelease  
Reading package lists... Done  
seun@seun-virtual-machine:~$
```

Step 2: Install required packages

```
sudo apt-get install ca-certificates curl gnupg
```

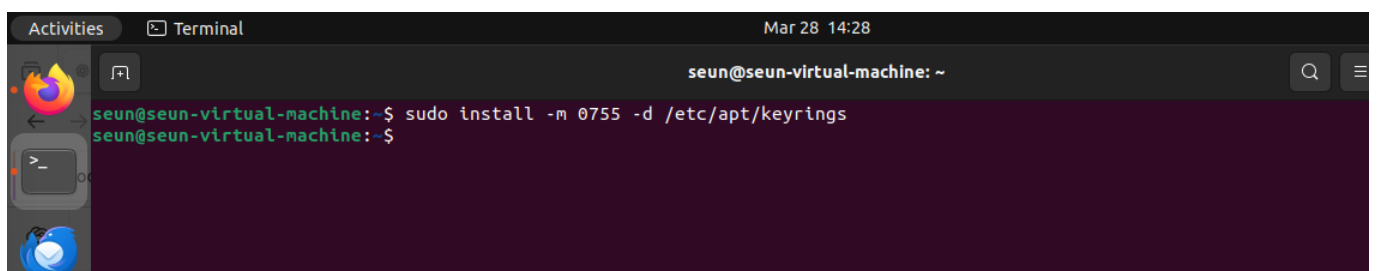


A terminal window titled 'Terminal' with the date 'Mar 28 14:26' in the top right. The prompt is 'seun@seun-virtual-machine: ~'. The user enters the command 'sudo apt-get install -y ca-certificates curl gnupg'. The terminal output shows the package lists being read, dependencies being built, and state information being read. It confirms that 'ca-certificates' is already the newest version (20240203-22.04.1) and 'gnupg' is already the newest version (2.2.27-3ubuntu2.5). It lists several packages that were automatically installed and are no longer required, including 'linux-headers-6.8.0-86-generic', 'linux-hwe-6.8-headers-6.8.0-86', 'linux-hwe-6.8-tools-6.8.0-86', 'linux-image-6.8.0-86-generic', 'linux-modules-6.8.0-86-generic', 'linux-modules-extra-6.8.0-86-generic', and 'linux-tools-6.8.0-86-generic'. It suggests using 'sudo apt autoremove' to remove them. It then lists the additional packages to be installed: 'libcurl4'. It lists the packages to be upgraded: 'curl' and 'libcurl4'. It shows the upgrade statistics: '2 upgraded, 0 newly installed, 0 to remove and 125 not upgraded'. It indicates that no space is needed for archives. It shows the disk space requirements and the files and directories currently installed. It then shows the preparation to unpack the curl and libcurl4 packages, followed by the unpacking process. It shows the setting up of the packages and the processing of triggers for 'man-db' and 'libc-bin'.

```
seun@seun-virtual-machine:~$ sudo apt-get install -y ca-certificates curl gnupg
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20240203-22.04.1).
gnupg is already the newest version (2.2.27-3ubuntu2.5).
gnupg set to manually installed.
The following packages were automatically installed and are no longer required:
  linux-headers-6.8.0-86-generic linux-hwe-6.8-headers-6.8.0-86 linux-hwe-6.8-tools-6.8.0-86 linux-image-6.8.0-86-generic
  linux-modules-6.8.0-86-generic linux-modules-extra-6.8.0-86-generic linux-tools-6.8.0-86-generic
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  libcurl4
The following packages will be upgraded:
  curl libcurl4
2 upgraded, 0 newly installed, 0 to remove and 125 not upgraded.
Need to get 0 B/484 kB of archives.
After this operation, 0 B of additional disk space will be used.
(Reading database ... 253006 files and directories currently installed.)
Preparing to unpack .../curl_7.81.0-1ubuntu1.23_amd64.deb ...
Unpacking curl (7.81.0-1ubuntu1.23) over (7.81.0-1ubuntu1.21) ...
Preparing to unpack .../libcurl4_7.81.0-1ubuntu1.23_amd64.deb ...
Unpacking libcurl4:amd64 (7.81.0-1ubuntu1.23) over (7.81.0-1ubuntu1.21) ...
Setting up libcurl4:amd64 (7.81.0-1ubuntu1.23) ...
Setting up curl (7.81.0-1ubuntu1.23) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for libc-bin (2.35-0ubuntu3.11) ...
seun@seun-virtual-machine:~$
```

Step 3: Create keyring directory

```
sudo install -m 0755 -d /etc/apt/keyrings
```

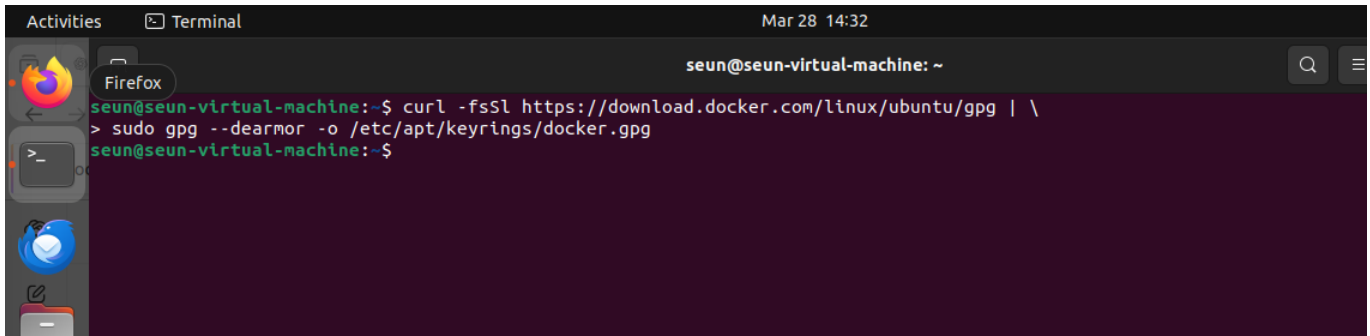


A terminal window titled 'Terminal' with the date 'Mar 28 14:28' in the top right. The prompt is 'seun@seun-virtual-machine: ~'. The user enters the command 'sudo install -m 0755 -d /etc/apt/keyrings'. The terminal output shows the command being executed successfully, and the prompt returns to 'seun@seun-virtual-machine:~\$'.

```
seun@seun-virtual-machine:~$ sudo install -m 0755 -d /etc/apt/keyrings
seun@seun-virtual-machine:~$
```

Step 4: Add Docker GPG key

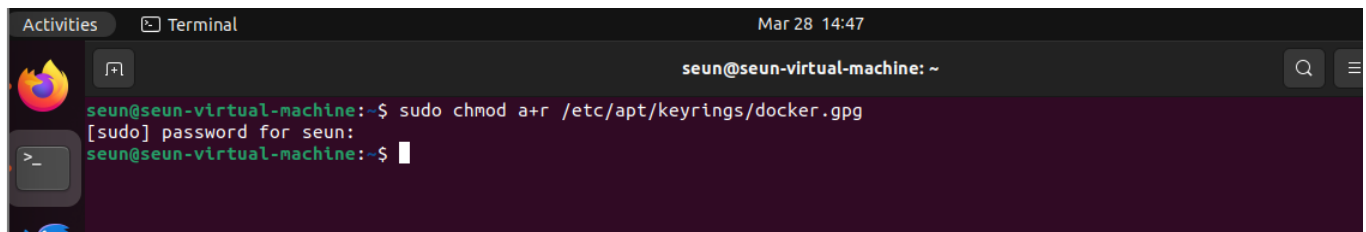
```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```



A terminal window titled 'Terminal' with the date 'Mar 28 14:32'. The prompt is 'seun@seun-virtual-machine: ~'. The user enters the command: `curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \` followed by `> sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg`. The prompt returns to `seun@seun-virtual-machine:~$`.

Step 5: Set permissions

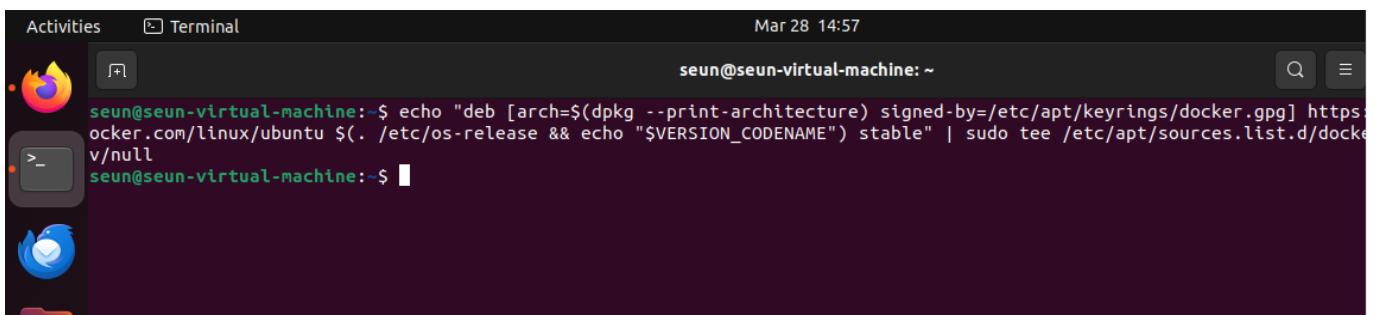
```
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```



A terminal window titled 'Terminal' with the date 'Mar 28 14:47'. The prompt is 'seun@seun-virtual-machine: ~'. The user enters the command: `sudo chmod a+r /etc/apt/keyrings/docker.gpg`. A password prompt appears: `[sudo] password for seun:`. The user enters a password (indicated by a cursor). The prompt returns to `seun@seun-virtual-machine:~$`.

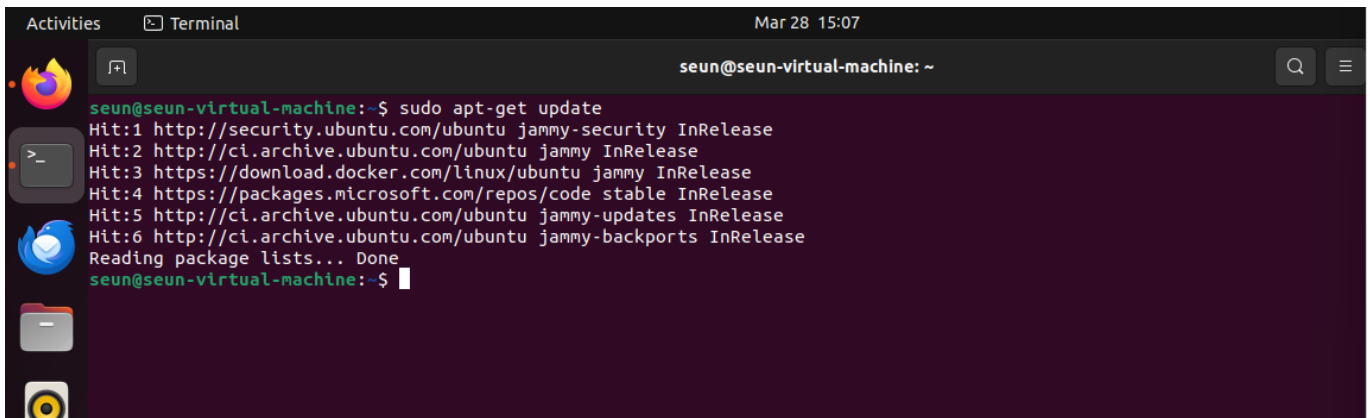
Step 6: Add Docker repository

```
echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```



A terminal window titled 'Terminal' with the date 'Mar 28 14:57'. The prompt is 'seun@seun-virtual-machine: ~'. The user enters the command: `echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null`. The prompt returns to `seun@seun-virtual-machine:~$`.

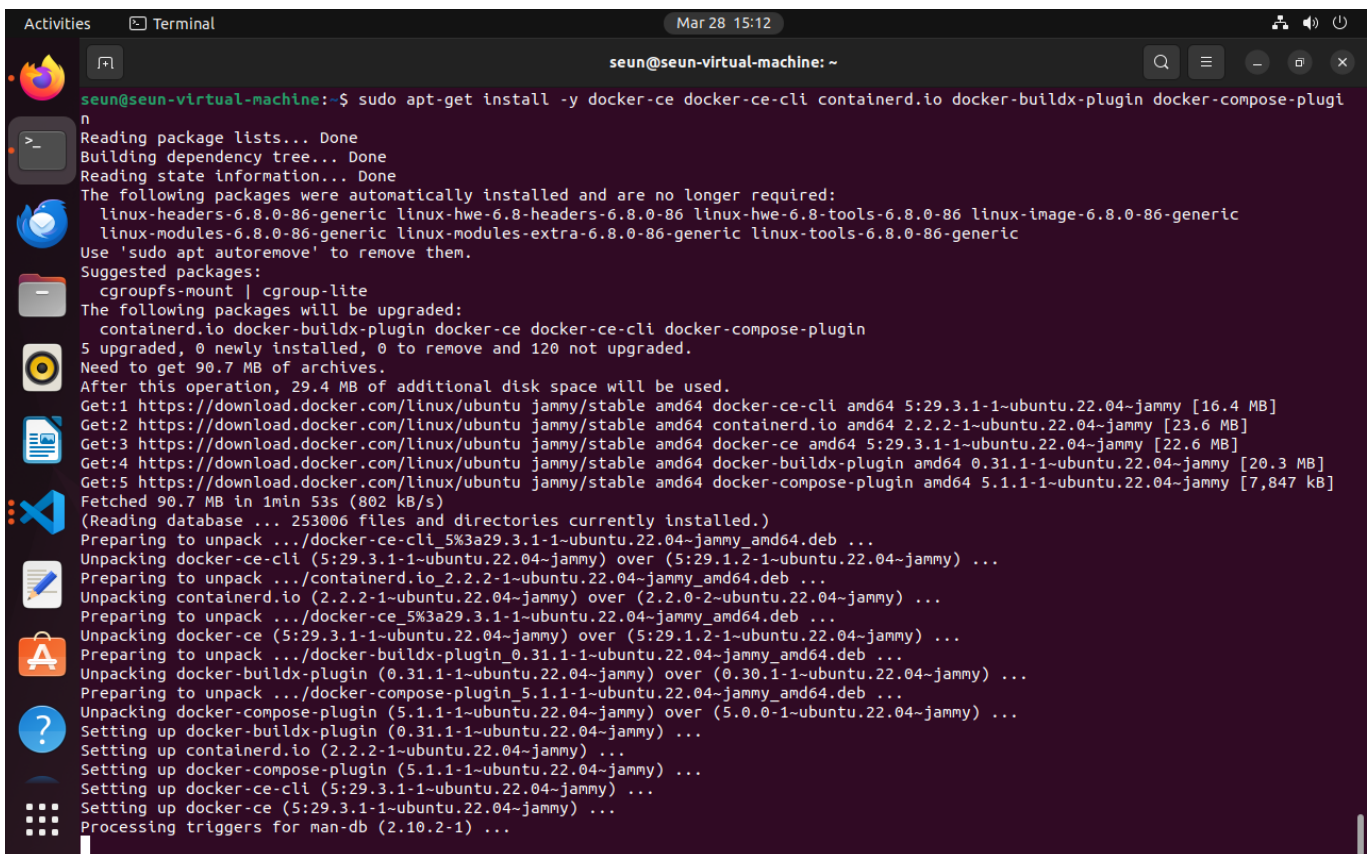
Step 7: Update package index again



```
seun@seun-virtual-machine:~$ sudo apt-get update
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://ci.archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 https://download.docker.com/linux/ubuntu jammy InRelease
Hit:4 https://packages.microsoft.com/repos/code stable InRelease
Hit:5 http://ci.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:6 http://ci.archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
seun@seun-virtual-machine:~$
```

Step 8: Install Docker

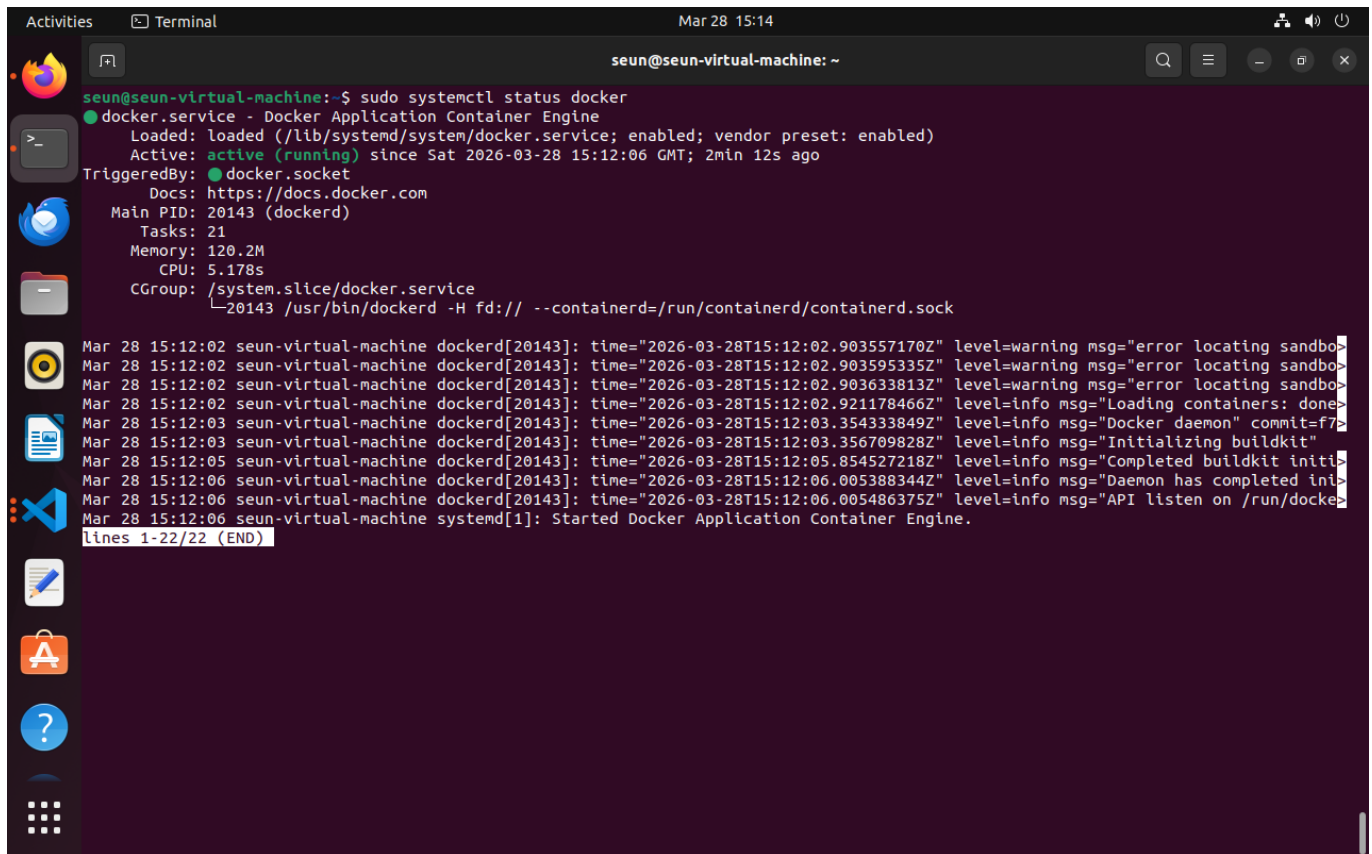
```
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```



```
seun@seun-virtual-machine:~$ sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugi
n
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  linux-headers-6.8.0-86-generic linux-hwe-6.8-headers-6.8.0-86 linux-hwe-6.8-tools-6.8.0-86 linux-image-6.8.0-86-generic
  linux-modules-6.8.0-86-generic linux-modules-extra-6.8.0-86-generic linux-tools-6.8.0-86-generic
Use 'sudo apt autoremove' to remove them.
Suggested packages:
  cgroupfs-mount | cgroup-lite
The following packages will be upgraded:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-compose-plugin
5 upgraded, 0 newly installed, 0 to remove and 120 not upgraded.
Need to get 90.7 MB of archives.
After this operation, 29.4 MB of additional disk space will be used.
Get:1 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-ce-cli amd64 5:29.3.1-1-ubuntu.22.04-jammy [16.4 MB]
Get:2 https://download.docker.com/linux/ubuntu jammy/stable amd64 containerd.io amd64 2.2.2-1-ubuntu.22.04-jammy [23.6 MB]
Get:3 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-ce amd64 5:29.3.1-1-ubuntu.22.04-jammy [22.6 MB]
Get:4 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-buildx-plugin amd64 0.31.1-1-ubuntu.22.04-jammy [20.3 MB]
Get:5 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-compose-plugin amd64 5.1.1-1-ubuntu.22.04-jammy [7,847 kB]
Fetched 90.7 MB in 1min 53s (802 kB/s)
(Reading database ... 253006 files and directories currently installed.)
Preparing to unpack .../docker-ce-cli_5%3a29.3.1-1-ubuntu.22.04-jammy_amd64.deb ...
Unpacking docker-ce-cli (5:29.3.1-1-ubuntu.22.04-jammy) over (5:29.1.2-1-ubuntu.22.04-jammy) ...
Preparing to unpack .../containerd.io_2.2.2-1-ubuntu.22.04-jammy_amd64.deb ...
Unpacking containerd.io (2.2.2-1-ubuntu.22.04-jammy) over (2.2.0-2-ubuntu.22.04-jammy) ...
Preparing to unpack .../docker-ce_5%3a29.3.1-1-ubuntu.22.04-jammy_amd64.deb ...
Unpacking docker-ce (5:29.3.1-1-ubuntu.22.04-jammy) over (5:29.1.2-1-ubuntu.22.04-jammy) ...
Preparing to unpack .../docker-buildx-plugin_0.31.1-1-ubuntu.22.04-jammy_amd64.deb ...
Unpacking docker-buildx-plugin (0.31.1-1-ubuntu.22.04-jammy) over (0.30.1-1-ubuntu.22.04-jammy) ...
Preparing to unpack .../docker-compose-plugin_5.1.1-1-ubuntu.22.04-jammy_amd64.deb ...
Unpacking docker-compose-plugin (5.1.1-1-ubuntu.22.04-jammy) over (5.0.0-1-ubuntu.22.04-jammy) ...
Setting up containerd.io (2.2.2-1-ubuntu.22.04-jammy) ...
Setting up docker-compose-plugin (5.1.1-1-ubuntu.22.04-jammy) ...
Setting up docker-ce-cli (5:29.3.1-1-ubuntu.22.04-jammy) ...
Setting up docker-ce (5:29.3.1-1-ubuntu.22.04-jammy) ...
Processing triggers for man-db (2.10.2-1) ...
```

Step 9: Verify Docker

```
sudo systemctl status docker
```



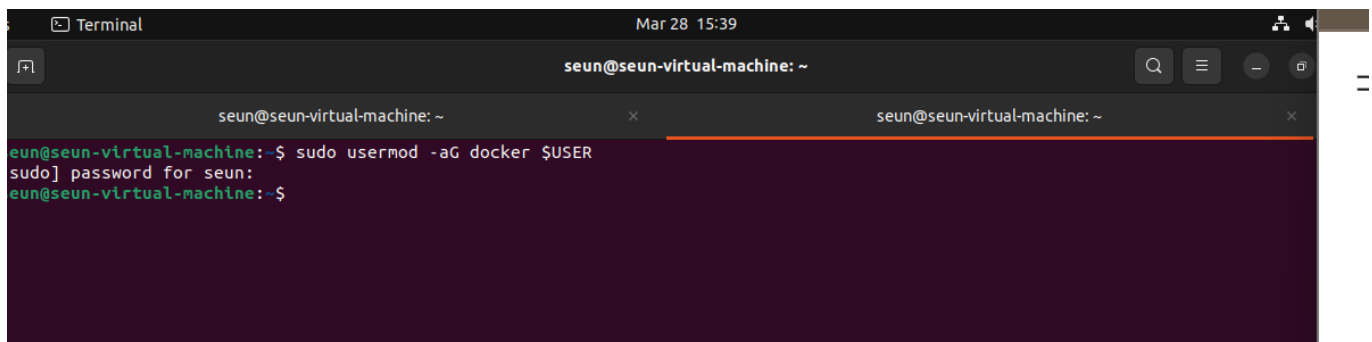
A terminal window titled 'seun@seun-virtual-machine: ~' showing the output of the command `sudo systemctl status docker`. The output indicates that the Docker service is active and running. Below the service status, there are several log entries from the Docker daemon (dockerd) and the systemd service, showing the initialization process and some warnings about sandboxing.

```
seun@seun-virtual-machine:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-03-28 15:12:06 GMT; 2min 12s ago
     TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
    Main PID: 20143 (dockerd)
      Tasks: 21
     Memory: 120.2M
        CPU: 5.178s
    CGroup: /system.slice/docker.service
            └─20143 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Mar 28 15:12:02 seun-virtual-machine dockerd[20143]: time="2026-03-28T15:12:02.903557170Z" level=warning msg="error locating sandbox"
Mar 28 15:12:02 seun-virtual-machine dockerd[20143]: time="2026-03-28T15:12:02.903595335Z" level=warning msg="error locating sandbox"
Mar 28 15:12:02 seun-virtual-machine dockerd[20143]: time="2026-03-28T15:12:02.903633813Z" level=warning msg="error locating sandbox"
Mar 28 15:12:02 seun-virtual-machine dockerd[20143]: time="2026-03-28T15:12:02.921178466Z" level=info msg="Loading containers: done"
Mar 28 15:12:03 seun-virtual-machine dockerd[20143]: time="2026-03-28T15:12:03.354333849Z" level=info msg="Docker daemon" commit=f7
Mar 28 15:12:03 seun-virtual-machine dockerd[20143]: time="2026-03-28T15:12:03.356709828Z" level=info msg="Initializing buildkit"
Mar 28 15:12:05 seun-virtual-machine dockerd[20143]: time="2026-03-28T15:12:05.854527218Z" level=info msg="Completed buildkit initia
Mar 28 15:12:06 seun-virtual-machine dockerd[20143]: time="2026-03-28T15:12:06.005388344Z" level=info msg="Daemon has completed ini
Mar 28 15:12:06 seun-virtual-machine dockerd[20143]: time="2026-03-28T15:12:06.005486375Z" level=info msg="API listen on /run/docke
Mar 28 15:12:06 seun-virtual-machine systemd[1]: Started Docker Application Container Engine.
lines 1-22/22 (END)
```

Step 10: Add user to Docker group

```
sudo usermod -aG docker $USER
```

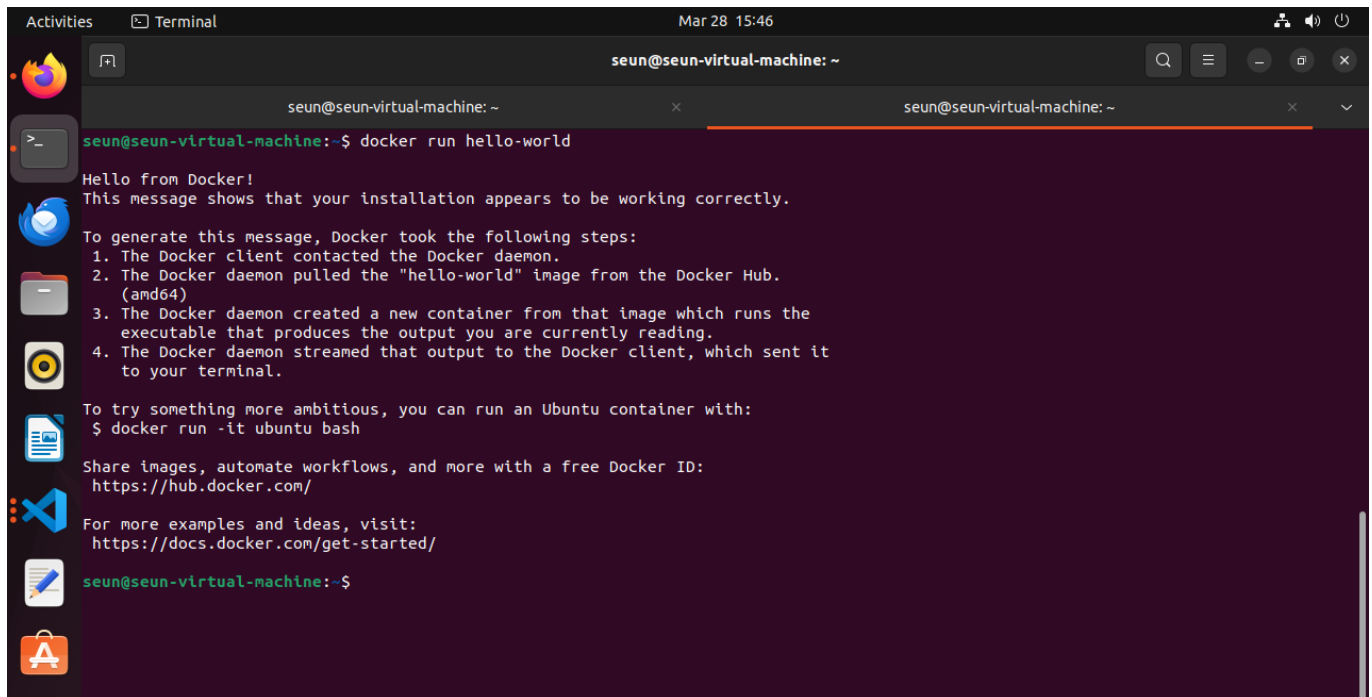


A terminal window titled 'seun@seun-virtual-machine: ~' showing the execution of the command `sudo usermod -aG docker $USER`. The prompt asks for the password for the user 'seun', which is entered and accepted.

```
seun@seun-virtual-machine:~$ sudo usermod -aG docker $USER
sudo] password for seun:
seun@seun-virtual-machine:~$
```

Step 11: Test Docker

```
docker run hello-world
```

A terminal window titled 'seun@seun-virtual-machine: ~' showing the command 'docker run hello-world' and its output. The output includes a 'Hello from Docker!' message, a confirmation that the installation is working, a list of steps Docker took (contacting daemon, pulling image, creating container, streaming output), and instructions to try running an Ubuntu container or share images.

```
seun@seun-virtual-machine:~$ docker run hello-world
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

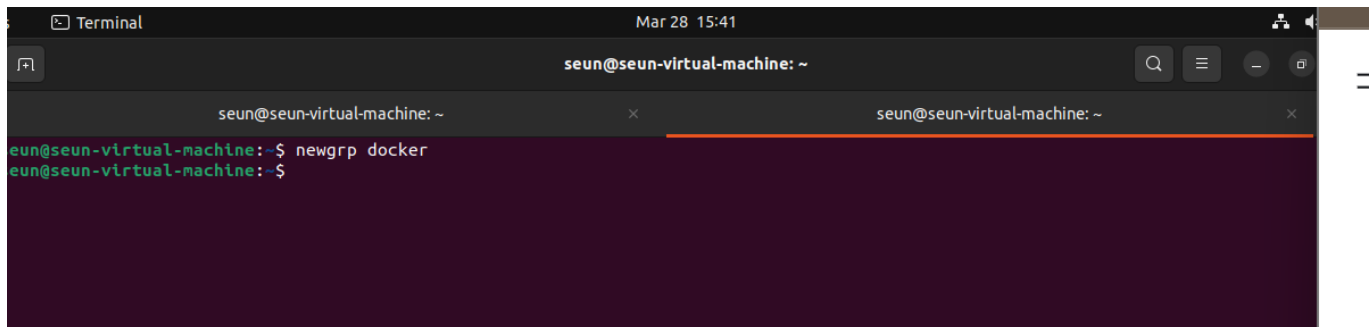
To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
seun@seun-virtual-machine:~$
```

Step 12: Apply group changes

```
newgrp docker
```

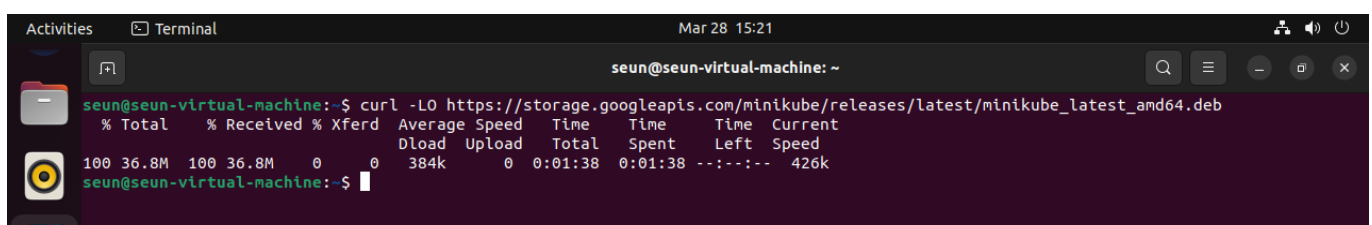
A terminal window titled 'seun@seun-virtual-machine: ~' showing the command 'newgrp docker' being executed. The prompt changes from 'seun@seun-virtual-machine:~\$' to 'seun@seun-virtual-machine:~\$' after the command is run.

```
seun@seun-virtual-machine:~$ newgrp docker
seun@seun-virtual-machine:~$
```

◆ PHASE 3: Install Minikube

Step 1: Download Minikube

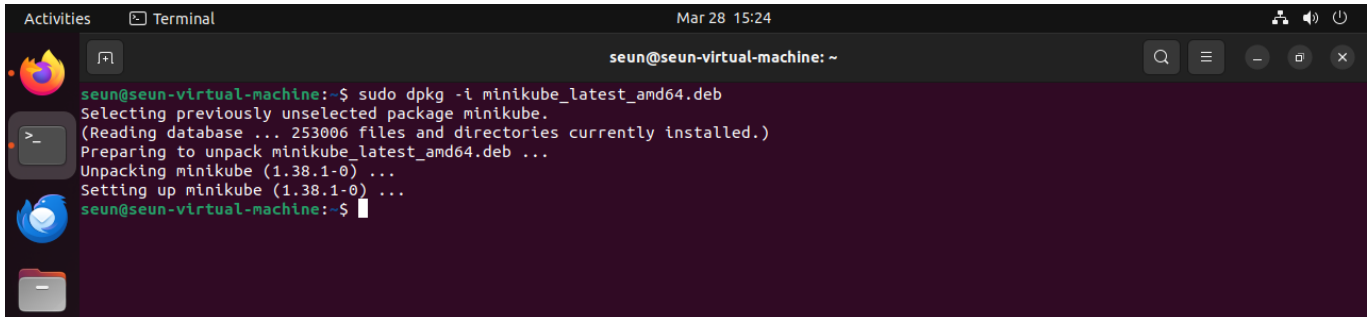
```
curl -LO
https://storage.googleapis.com/minikube/releases/latest/minikube_latest_amd64.deb
```

A terminal window titled 'seun@seun-virtual-machine: ~' showing the command 'curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube_latest_amd64.deb' being executed. The output shows the download progress and completion.

```
seun@seun-virtual-machine:~$ curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube_latest_amd64.deb
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 36.8M 100 36.8M 0 0 384k 0 0:01:38 0:01:38 --:--:-- 426k
seun@seun-virtual-machine:~$
```

Step 2: Install Minikube

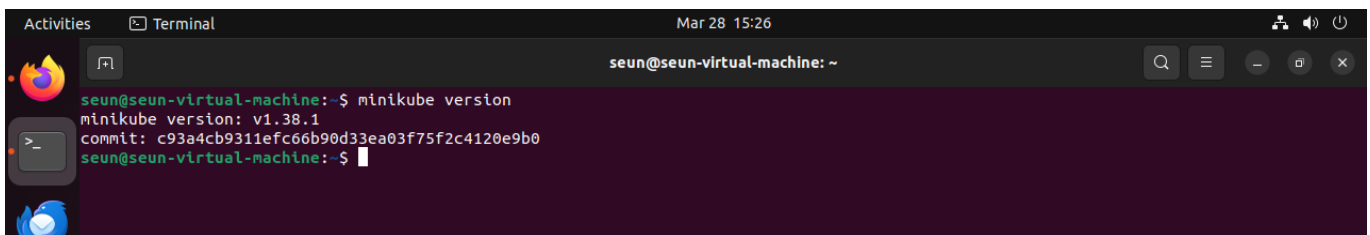
```
sudo dpkg -i minikube_latest_amd64.deb
```

A terminal window titled 'Terminal' with the date 'Mar 28 15:24'. The prompt is 'seun@seun-virtual-machine: ~'. The user enters 'sudo dpkg -i minikube_latest_amd64.deb'. The output shows: 'Selecting previously unselected package minikube. (Reading database ... 253006 files and directories currently installed.) Preparing to unpack minikube_latest_amd64.deb ... Unpacking minikube (1.38.1-0) ... Setting up minikube (1.38.1-0) ... seun@seun-virtual-machine:~\$'.

```
seun@seun-virtual-machine:~$ sudo dpkg -i minikube_latest_amd64.deb
Selecting previously unselected package minikube.
(Reading database ... 253006 files and directories currently installed.)
Preparing to unpack minikube_latest_amd64.deb ...
Unpacking minikube (1.38.1-0) ...
Setting up minikube (1.38.1-0) ...
seun@seun-virtual-machine:~$
```

Step 3: Verify installation

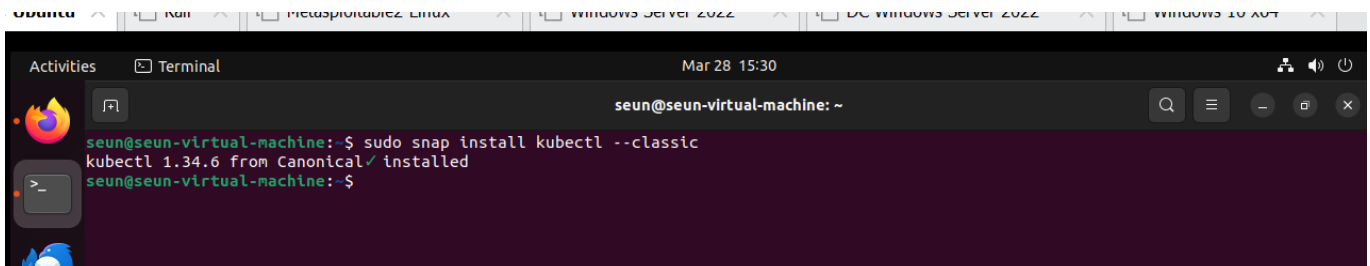
```
minikube version
```

A terminal window titled 'Terminal' with the date 'Mar 28 15:26'. The prompt is 'seun@seun-virtual-machine: ~'. The user enters 'minikube version'. The output shows: 'minikube version: v1.38.1 commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0 seun@seun-virtual-machine:~\$'.

```
seun@seun-virtual-machine:~$ minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
seun@seun-virtual-machine:~$
```

◆ PHASE 4: Install kubectl

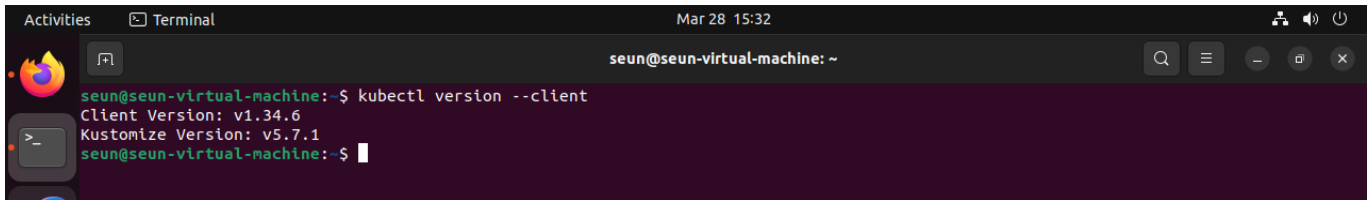
```
sudo snap install kubectl --classic
```

A terminal window titled 'Terminal' with the date 'Mar 28 15:30'. The prompt is 'seun@seun-virtual-machine: ~'. The user enters 'sudo snap install kubectl --classic'. The output shows: 'kubectl 1.34.6 from Canonical✓ installed seun@seun-virtual-machine:~\$'.

```
seun@seun-virtual-machine:~$ sudo snap install kubectl --classic
kubectl 1.34.6 from Canonical✓ installed
seun@seun-virtual-machine:~$
```

Verify kubectl

```
kubectl version --client
```

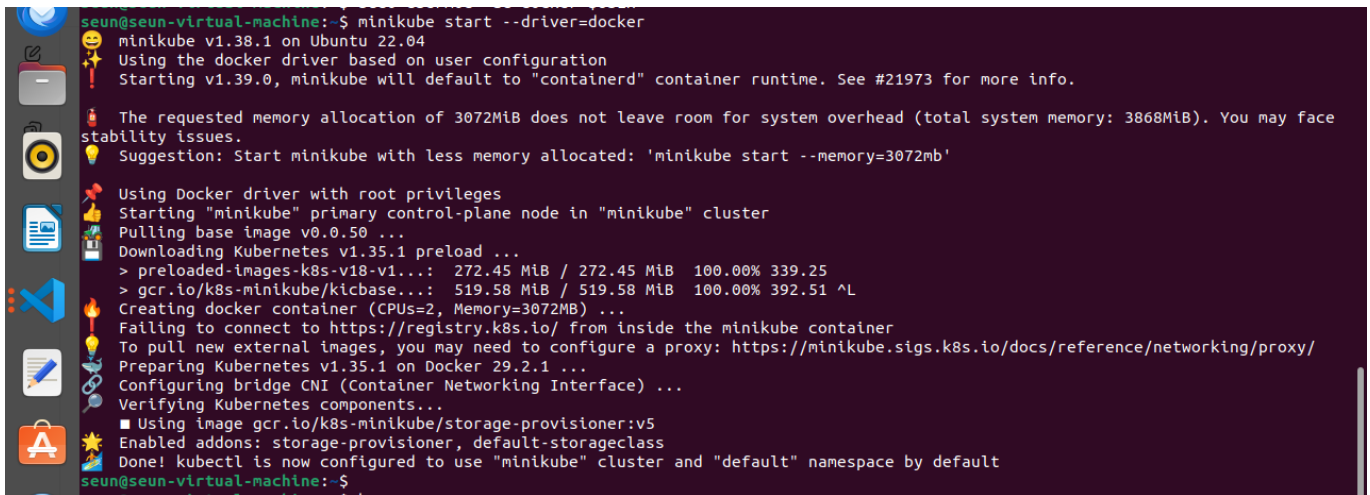



```
seun@seun-virtual-machine:~$ kubectl version --client
Client Version: v1.34.6
Kustomize Version: v5.7.1
seun@seun-virtual-machine:~$
```

◆ PHASE 5: Start Kubernetes Cluster

Step 1: Start Minikube

```
minikube start --driver=docker
```



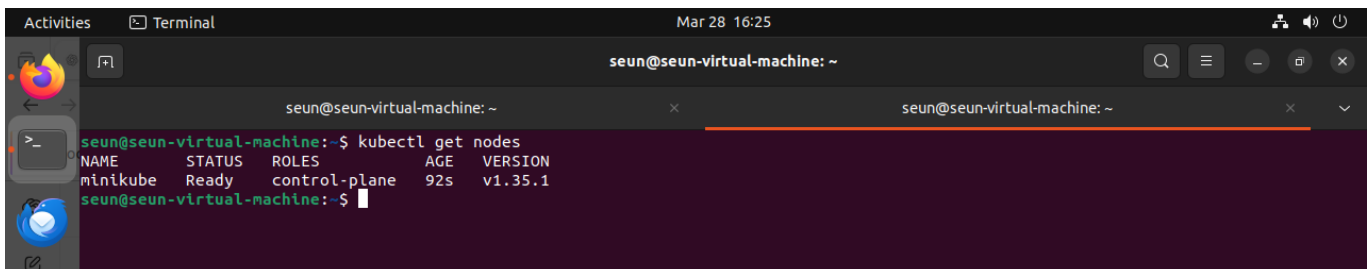
```
seun@seun-virtual-machine:~$ minikube start --driver=docker
minikube v1.38.1 on Ubuntu 22.04
Using the docker driver based on user configuration
Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.

! The requested memory allocation of 3072MiB does not leave room for system overhead (total system memory: 3868MiB). You may face
  stability issues.
! Suggestion: Start minikube with less memory allocated: 'minikube start --memory=3072mb'

Using Docker driver with root privileges
Starting "minikube" primary control-plane node in "minikube" cluster
Pulling base image v0.0.50 ...
Downloading Kubernetes v1.35.1 preload ...
> preloaded-images-k8s-v18-v1...: 272.45 MiB / 272.45 MiB 100.00% 339.25
> gcr.io/k8s-minikube/kicbase...: 519.58 MiB / 519.58 MiB 100.00% 392.51 ^L
Creating docker container (CPUs=2, Memory=3072MB) ...
Falling to connect to https://registry.k8s.io/ from inside the minikube container
To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networking/proxy/
Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
Configuring bridge CNI (Container Networking Interface) ...
Verifying Kubernetes components...
  ■ Using image gcr.io/k8s-minikube/storage-provisioner:v5
  Enabled addons: storage-provisioner, default-storageclass
Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
seun@seun-virtual-machine:~$
```

Step 2: Verify node

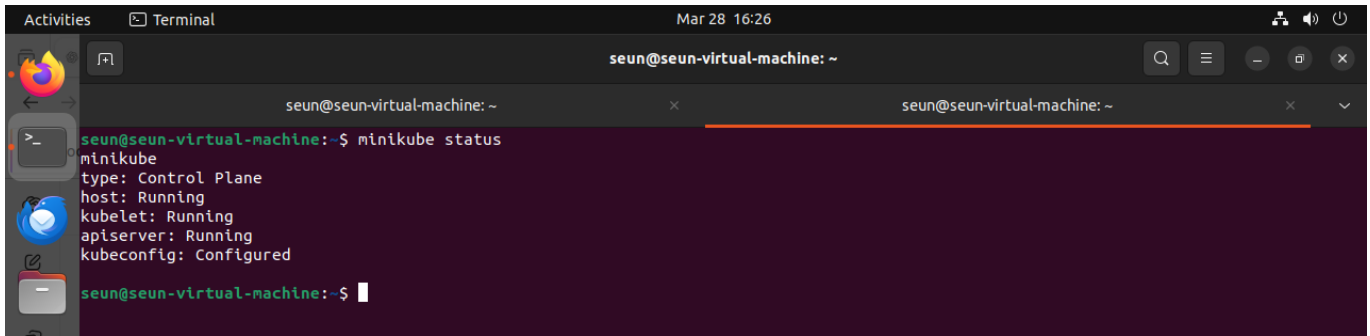
```
kubectl get nodes
```



```
seun@seun-virtual-machine:~$ kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready     control-plane  92s   v1.35.1
seun@seun-virtual-machine:~$
```

Step 3: Check Minikube status

```
minikube status
```

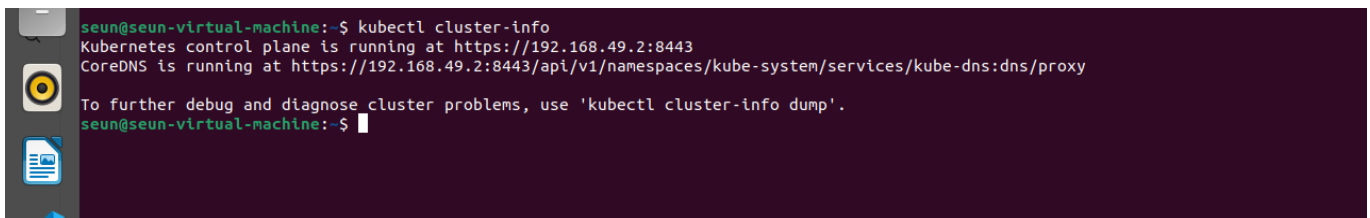


A terminal window titled 'seun@seun-virtual-machine: ~' showing the output of the 'minikube status' command. The output indicates that the minikube Control Plane, kubelet, apiserver, and kubeconfig are all in a 'Running' or 'Configured' state.

```
seun@seun-virtual-machine:~$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
seun@seun-virtual-machine:~$
```

Step 4: Cluster info

```
kubectl cluster-info
```



A terminal window showing the output of the 'kubectl cluster-info' command. It provides details about the Kubernetes control plane and CoreDNS, including their respective URLs.

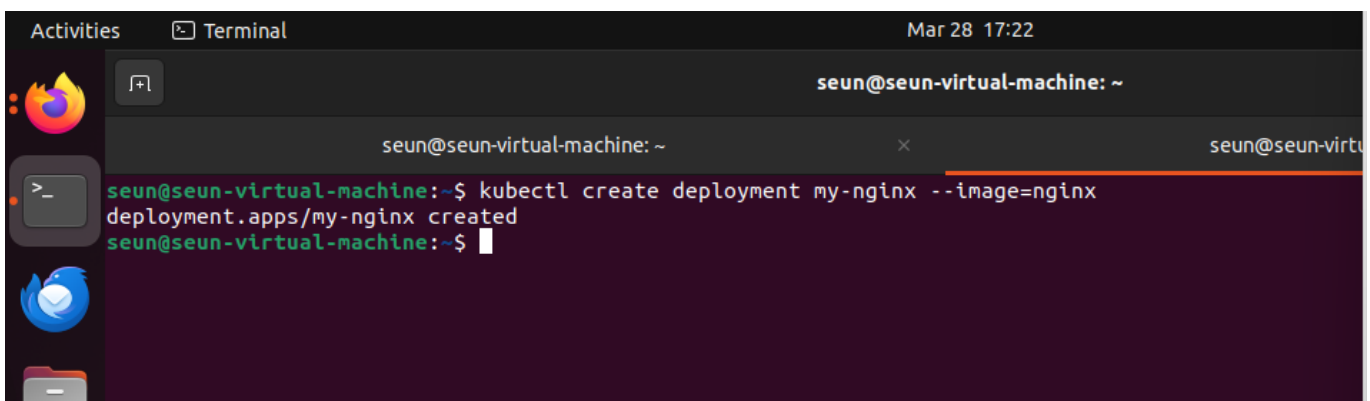
```
seun@seun-virtual-machine:~$ kubectl cluster-info
Kubernetes control plane is running at https://192.168.49.2:8443
CoreDNS is running at https://192.168.49.2:8443/api/v1/namespaces/kube-system/services/kube-dns/dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
seun@seun-virtual-machine:~$
```

◆ PHASE 6: Deploy Application

Step 1: Create deployment

```
kubectl create deployment my-nginx --image=nginx
```

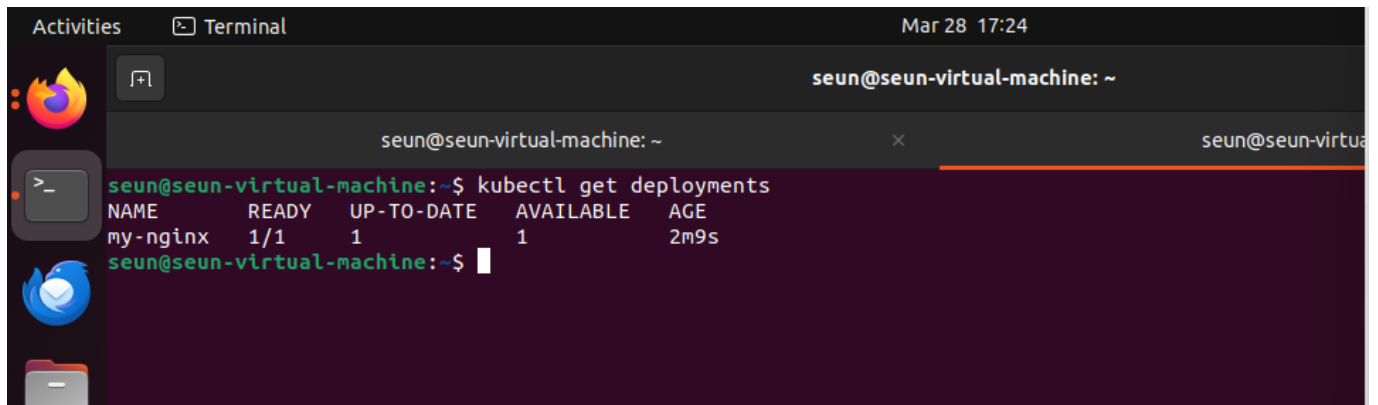


A terminal window showing the successful execution of the 'kubectl create deployment my-nginx --image=nginx' command. The output confirms that the deployment 'my-nginx' has been created in the 'apps' namespace.

```
seun@seun-virtual-machine:~$ kubectl create deployment my-nginx --image=nginx
deployment.apps/my-nginx created
seun@seun-virtual-machine:~$
```

Step 2: Verify deployment

```
kubectl get deployments
```

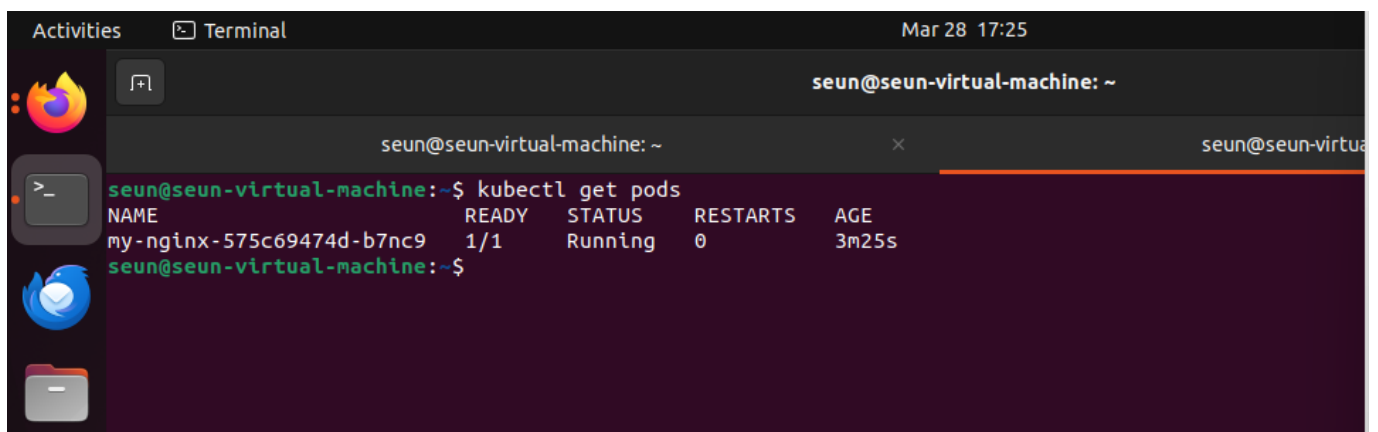


A terminal window titled 'seun@seun-virtual-machine: ~' showing the command 'kubectl get deployments' and its output. The output is a table with columns: NAME, READY, UP-TO-DATE, AVAILABLE, and AGE. The table contains one row for 'my-nginx'.

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
my-nginx	1/1	1	1	2m9s

Step 3: Check pods

```
kubectl get pods
```

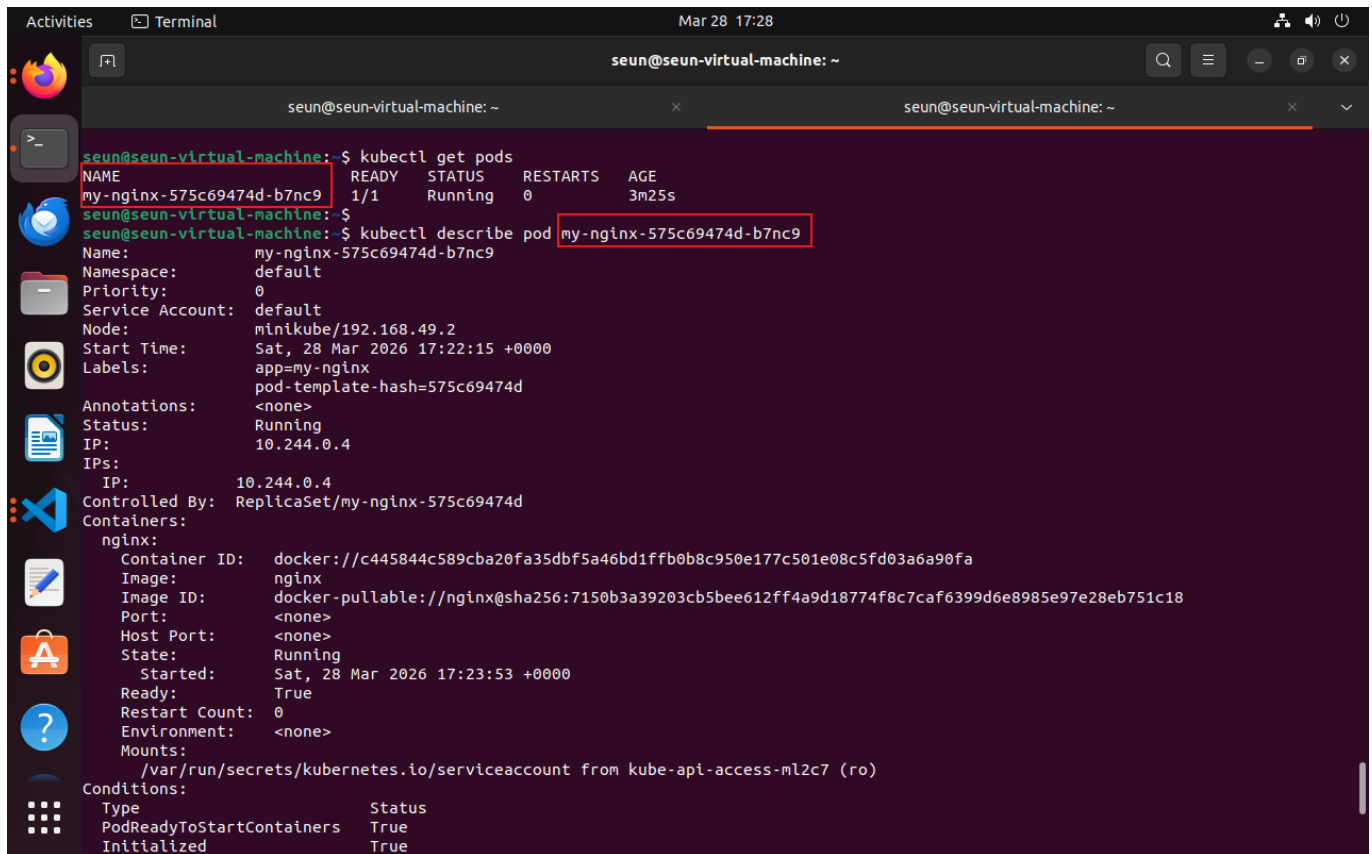


A terminal window titled 'seun@seun-virtual-machine: ~' showing the command 'kubectl get pods' and its output. The output is a table with columns: NAME, READY, STATUS, RESTARTS, and AGE. The table contains one row for 'my-nginx-575c69474d-b7nc9'.

NAME	READY	STATUS	RESTARTS	AGE
my-nginx-575c69474d-b7nc9	1/1	Running	0	3m25s

Step 4: Describe pod

```
kubectl describe pod <pod-name>
```

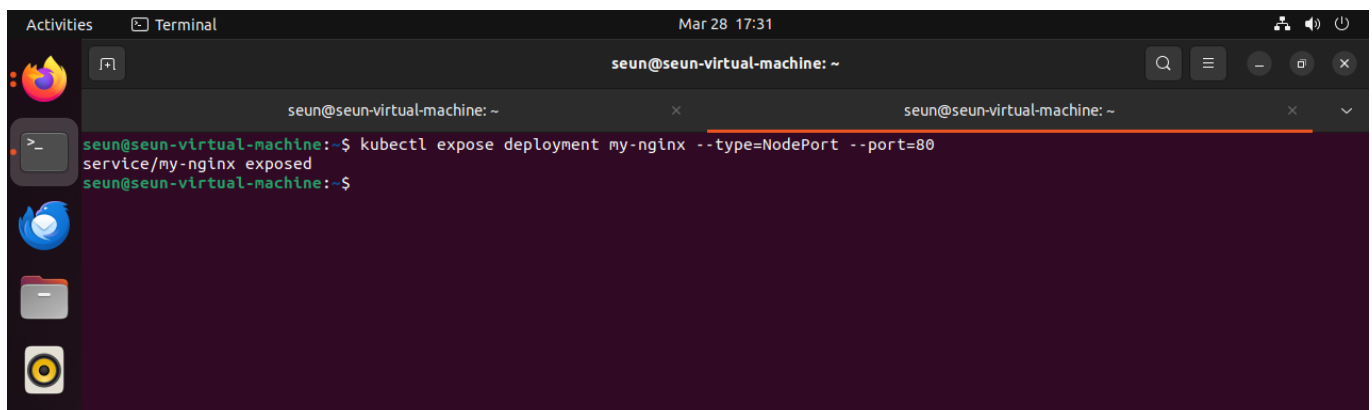


```
seun@seun-virtual-machine: ~  
$ kubectl get pods  
NAME                                READY  STATUS   RESTARTS  AGE  
my-nginx-575c69474d-b7nc9          1/1    Running   0          3m25s  
seun@seun-virtual-machine: ~  
$ kubectl describe pod my-nginx-575c69474d-b7nc9  
Name:                                my-nginx-575c69474d-b7nc9  
Namespace:                          default  
Priority:                             0  
Service Account:                     default  
Node:                                minikube/192.168.49.2  
Start Time:                          Sat, 28 Mar 2026 17:22:15 +0000  
Labels:                              app=my-nginx  
                                      pod-template-hash=575c69474d  
Annotations:                         <none>  
Status:                              Running  
IP:                                  10.244.0.4  
IPs:                                  IP: 10.244.0.4  
Controlled By:                       ReplicaSet/my-nginx-575c69474d  
Containers:  
  nginx:  
    Container ID:                     docker://c445844c589cba20fa35dbf5a46bd1ffb0b8c950e177c501e08c5fd03a6a90fa  
    Image:                           nginx  
    Image ID:                         docker-pullable://nginx@sha256:7150b3a39203cb5bee612ff4a9d18774f8c7caf6399d6e8985e97e28eb751c18  
    Port:                             <none>  
    Host Port:                        <none>  
    State:                            Running  
      Started:                        Sat, 28 Mar 2026 17:23:53 +0000  
    Ready:                            True  
    Restart Count:                     0  
    Environment:                     <none>  
    Mounts:                           /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-ml2c7 (ro)  
Conditions:  
  Type                               Status  
  PodReadyToStartContainers          True  
  Initialized                         True
```

◆ PHASE 7: Expose Application

Step 1: Expose deployment

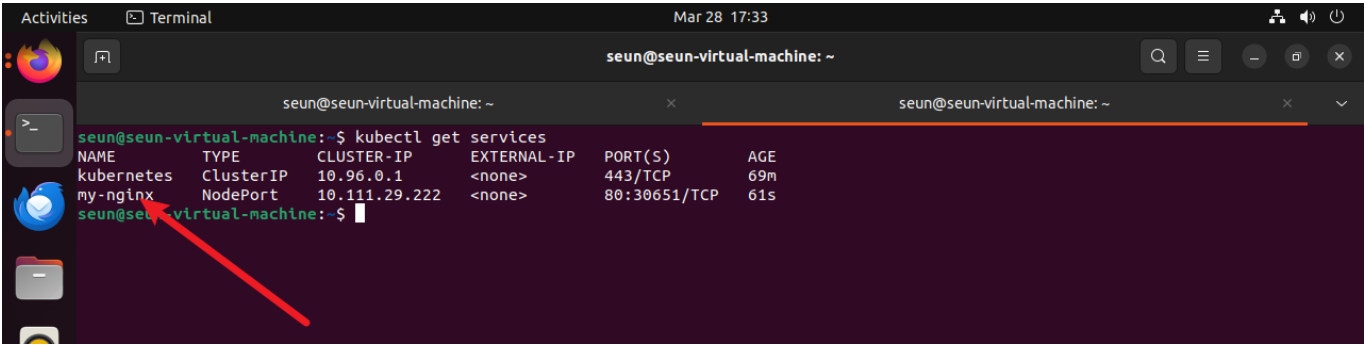
```
kubectl expose deployment my-nginx --type=NodePort --port=80
```



```
seun@seun-virtual-machine: ~  
$ kubectl expose deployment my-nginx --type=NodePort --port=80  
service/my-nginx exposed  
seun@seun-virtual-machine: ~$
```

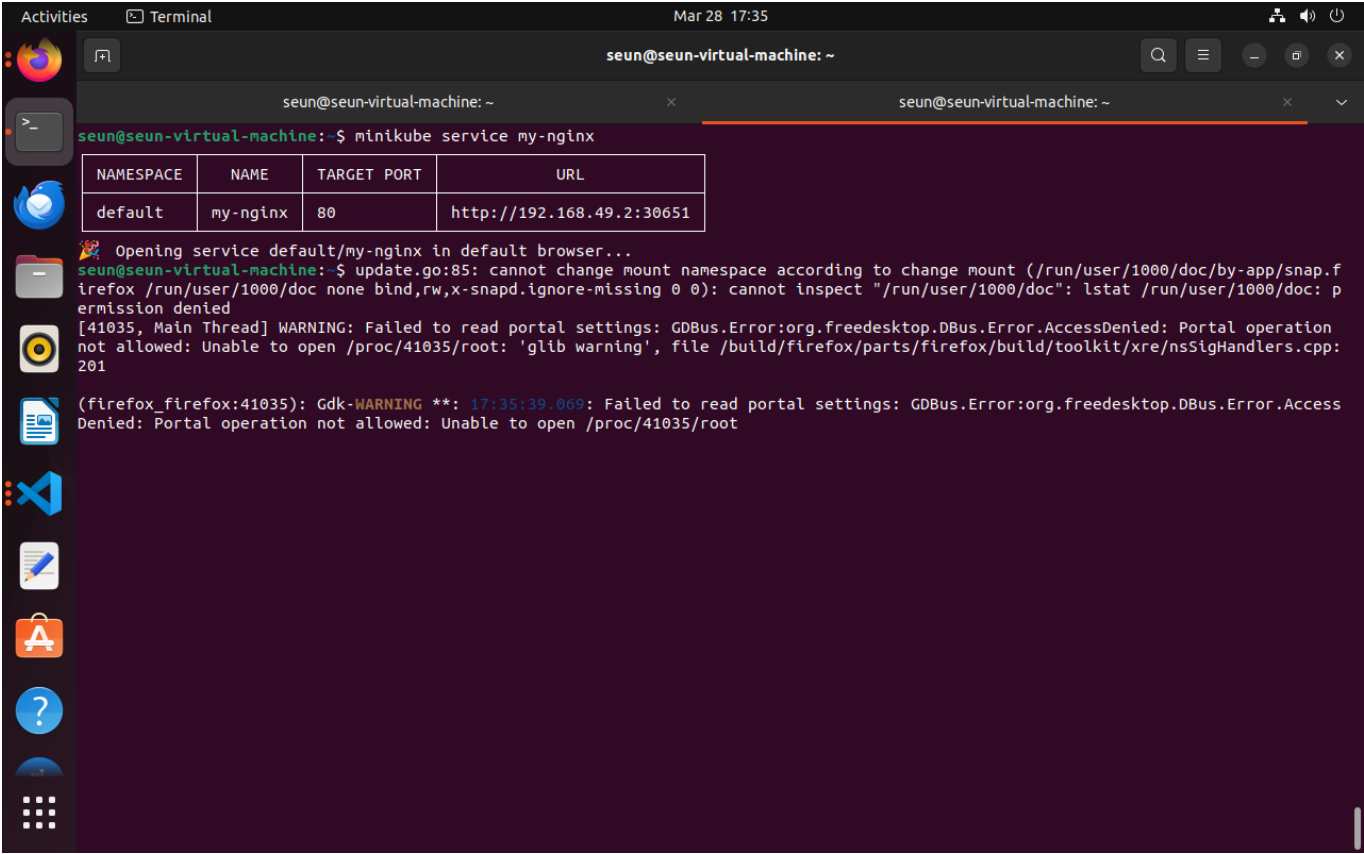
Step 2: Verify service

```
kubectl get services
```

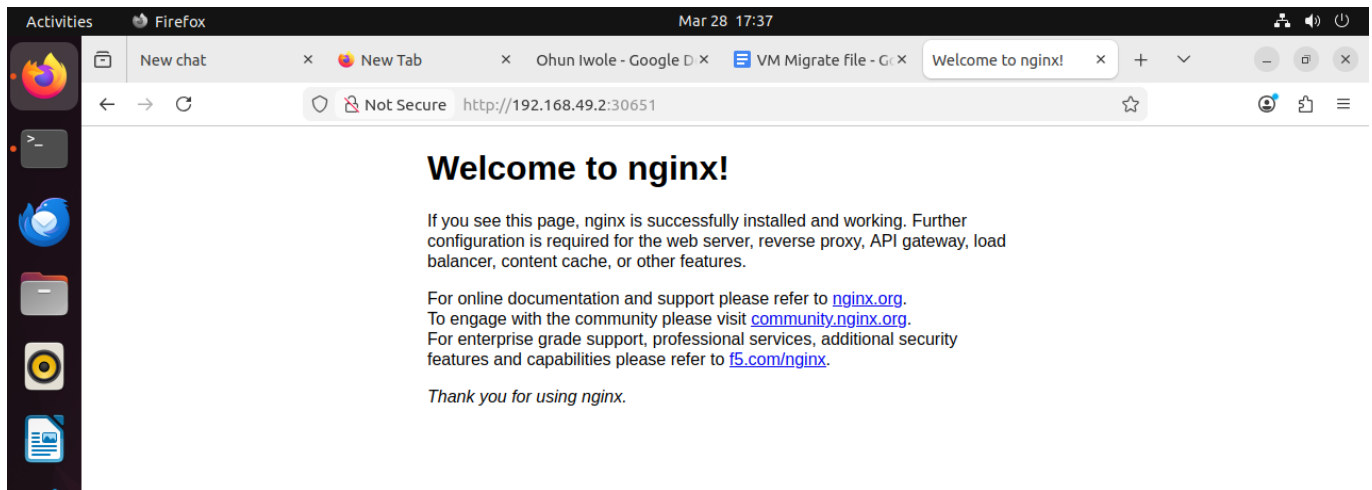


Step 3: Access application

```
minikube service my-nginx
```



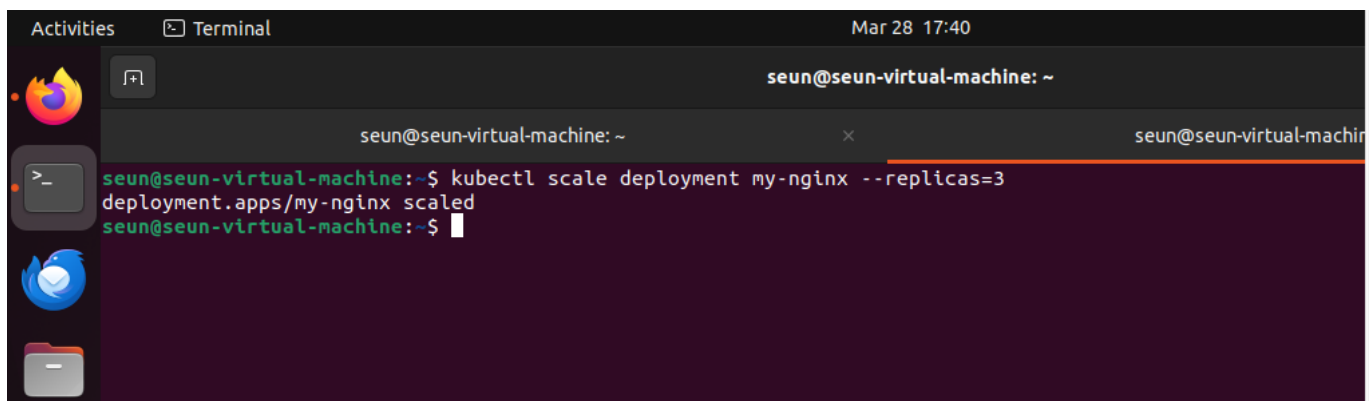
Step 4: Application in browser



◆ PHASE 8: Scaling & Self-Healing

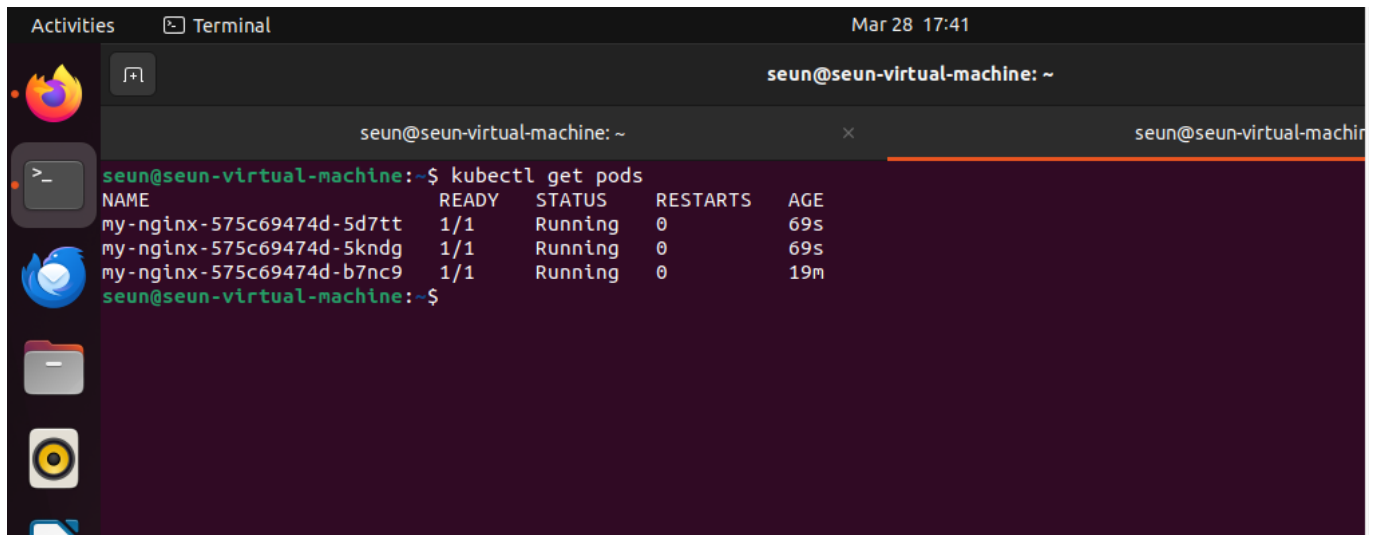
Step 1: Scale application

```
kubectl scale deployment my-nginx --replicas=3
```



Step 2: Verify pods

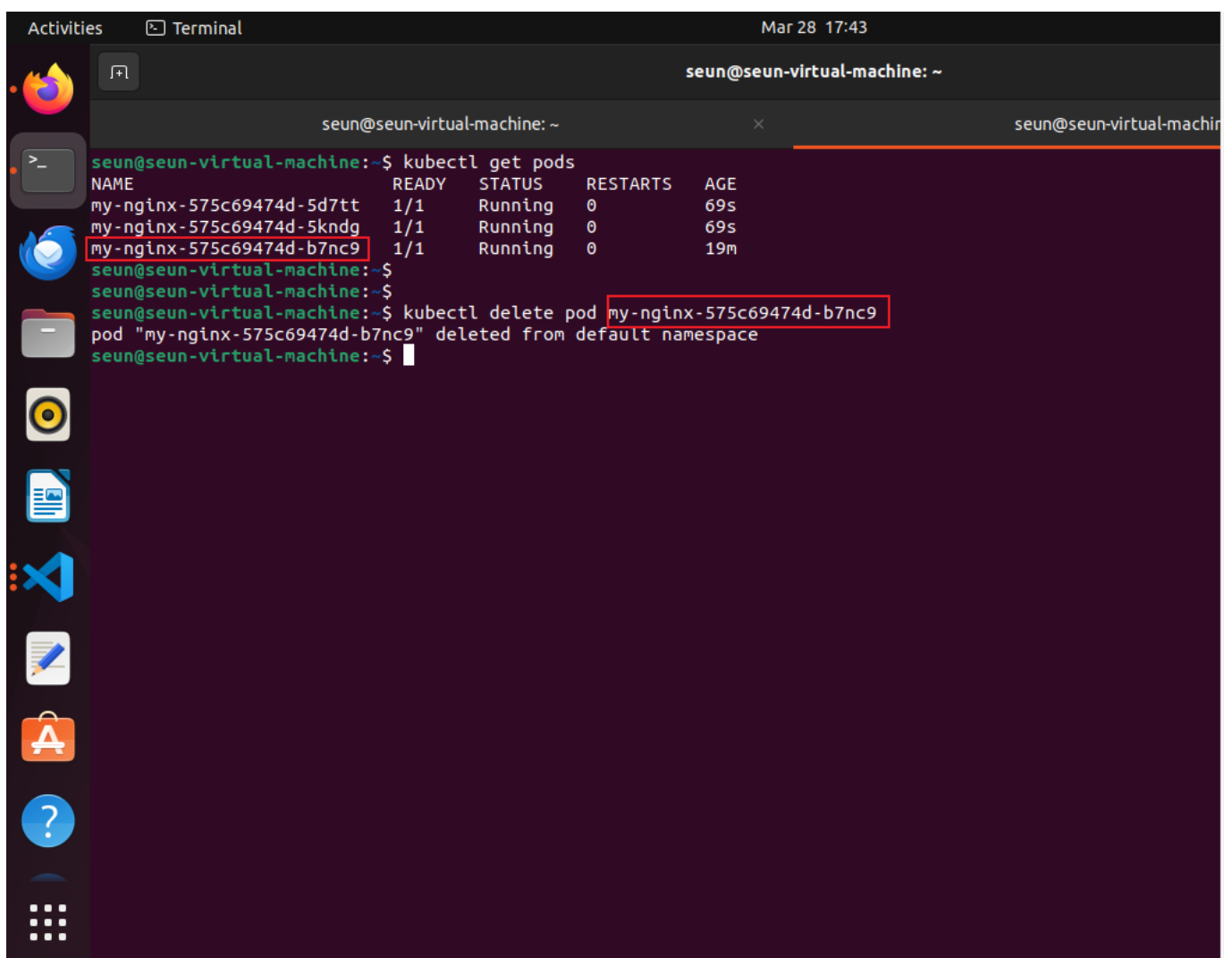
```
kubectl get pods
```

A terminal window titled 'seun@seun-virtual-machine: ~' showing the output of the 'kubectl get pods' command. The output is a table with columns: NAME, READY, STATUS, RESTARTS, and AGE. Three pods are listed, all in 'Running' status. The first two pods are 'my-nginx-575c69474d-5d7tt' and 'my-nginx-575c69474d-5kndg', both with 1/1 ready and 69s age. The third pod is 'my-nginx-575c69474d-b7nc9' with 1/1 ready and 19m age.

NAME	READY	STATUS	RESTARTS	AGE
my-nginx-575c69474d-5d7tt	1/1	Running	0	69s
my-nginx-575c69474d-5kndg	1/1	Running	0	69s
my-nginx-575c69474d-b7nc9	1/1	Running	0	19m

Step 3: Delete a pod

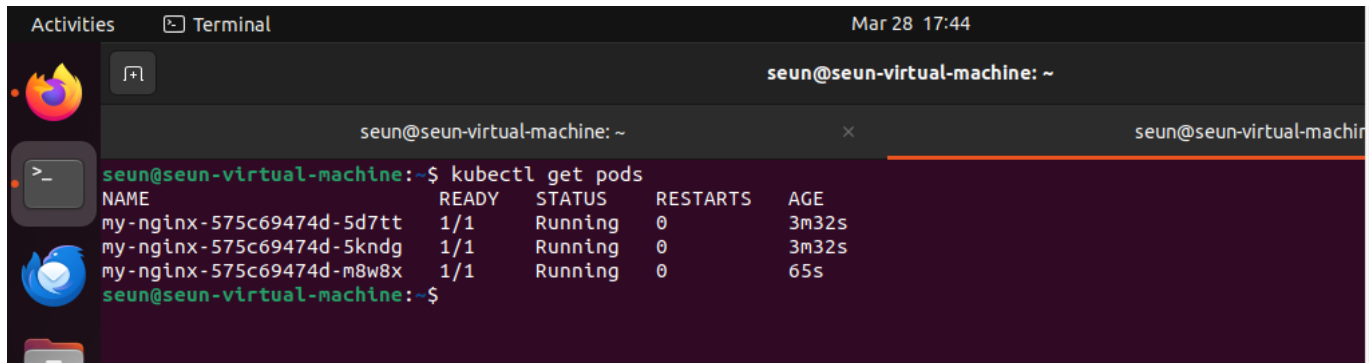
```
kubectl delete pod <pod-name>
```

A terminal window titled 'seun@seun-virtual-machine: ~' showing the deletion of a pod. It first runs 'kubectl get pods' showing the same three pods as before. Then, it runs 'kubectl delete pod my-nginx-575c69474d-b7nc9'. The output shows the pod being deleted from the default namespace.

```
seun@seun-virtual-machine:~$ kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
my-nginx-575c69474d-5d7tt          1/1     Running   0           69s
my-nginx-575c69474d-5kndg          1/1     Running   0           69s
my-nginx-575c69474d-b7nc9          1/1     Running   0           19m
seun@seun-virtual-machine:~$
seun@seun-virtual-machine:~$ kubectl delete pod my-nginx-575c69474d-b7nc9
pod "my-nginx-575c69474d-b7nc9" deleted from default namespace
seun@seun-virtual-machine:~$
```

Step 4: Observe self-healing

```
kubectl get pods
```



```
seun@seun-virtual-machine: ~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
my-nginx-575c69474d-5d7tt          1/1     Running   0           3m32s
my-nginx-575c69474d-5kndg          1/1     Running   0           3m32s
my-nginx-575c69474d-m8w8x          1/1     Running   0           65s
seun@seun-virtual-machine: ~$
```

Key Concepts Demonstrated

- Containerization using Docker
- Kubernetes cluster setup with Minikube
- Deployment and management of Pods
- Service exposure using NodePort
- Horizontal scaling
- Self-healing (automatic pod recreation)

Conclusion

This project successfully demonstrates how Kubernetes orchestrates containerized applications by automating deployment, scaling, and recovery. Using Minikube enabled a local simulation of a production-like Kubernetes environment.